

Fiddler. Настройка правил для подмены трафика на тестовых стендах

Fiddler – это прокси-сервер, который работает с трафиком между вашим компьютером и удаленным сервером, позволяя инспектировать и менять его. С помощью Fiddler можно делать подмену файлов, которые запрашивает браузер.



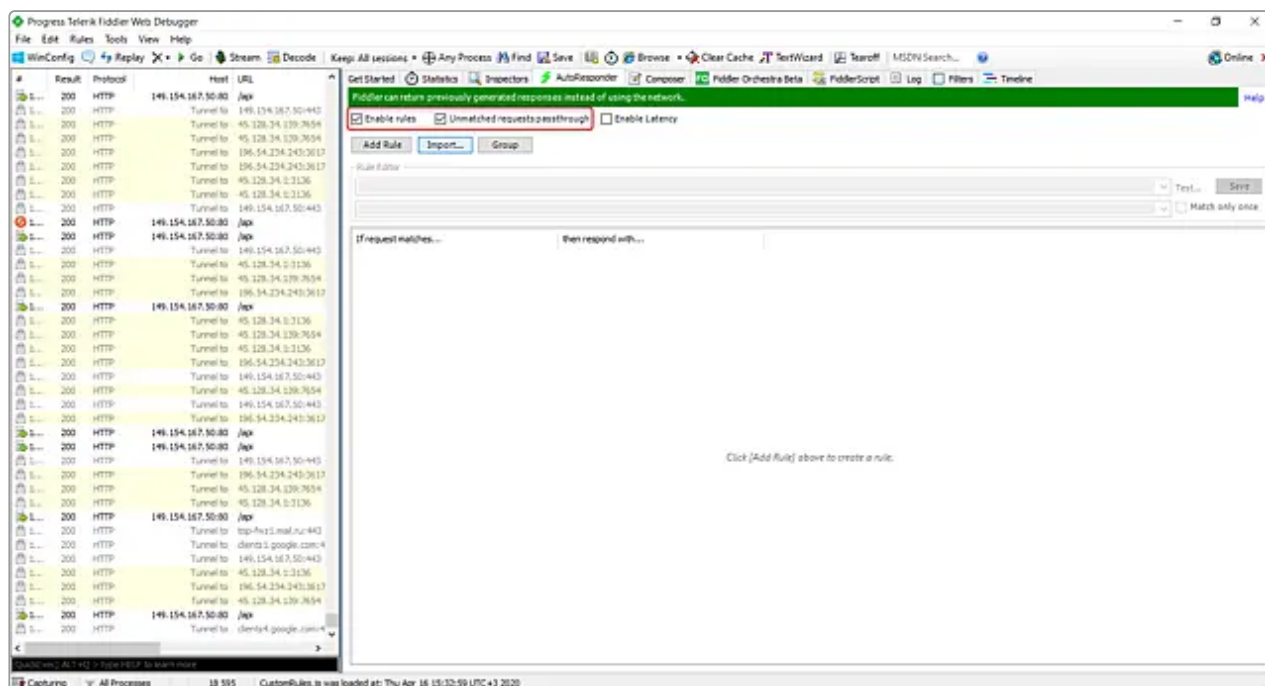
Настройка правил для подмены

1. Установите Fiddler и настройте для работы с https по инструкции:

- [Windows](#)
- [RedHat](#)

Далее запустите его.

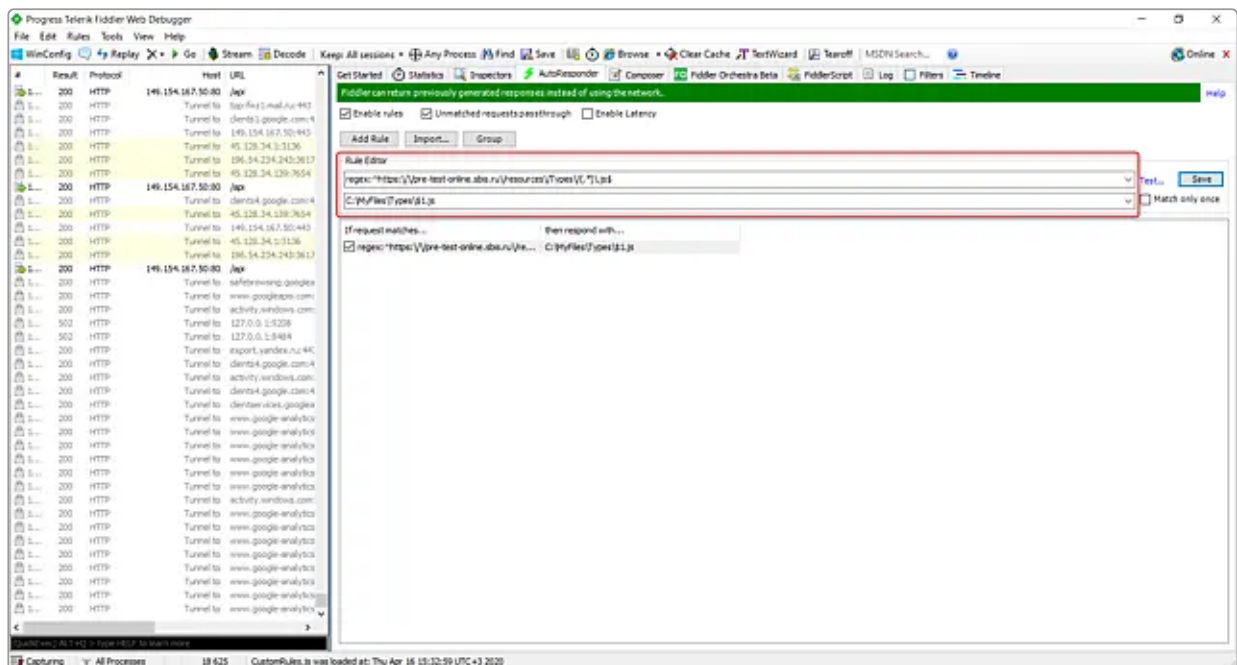
2. Перейдите на вкладку **AutoResponder** и отметьте флаги "Enable rules" и "Unmatched requests".



3. Нажмите кнопку "Add rule" для добавления правила.

4. Заполните поля **If request matches** и **then respond with** – по этим данным будет происходить подмена файла с сервера на какой-то, например, локальный файл.

- В поле **If request matches** может быть регулярное выражение. Чтобы значение трактовалось как регулярное выражение, нужно перед ним написать **regex**.
- В поле **then respond with** может быть адрес в формате url или путь к файлу на диске.



При использовании регулярных выражений можно отдельные части url из **If request matches** передавать в **then respond with**, используя в поле **then respond with** выражение **\$<число>**.

Например, если:

```
If request matches == regex:^https://test-
(online|cdn).sbis.ru/static/resources/Types/(.*)\\.js(?:.*)?$
then respond with == C:\\MyFiles\\Types$2.js
```

то при запросе:

```
https://test-online.sbis.ru/static/resources/Types/_entity/Model.js
https://test-cdn.sbis.ru/static/resources/Types/_entity/Model.js
```

будет отдано содержимое файла C:\\MyFiles\\Types_entity\\Model.js

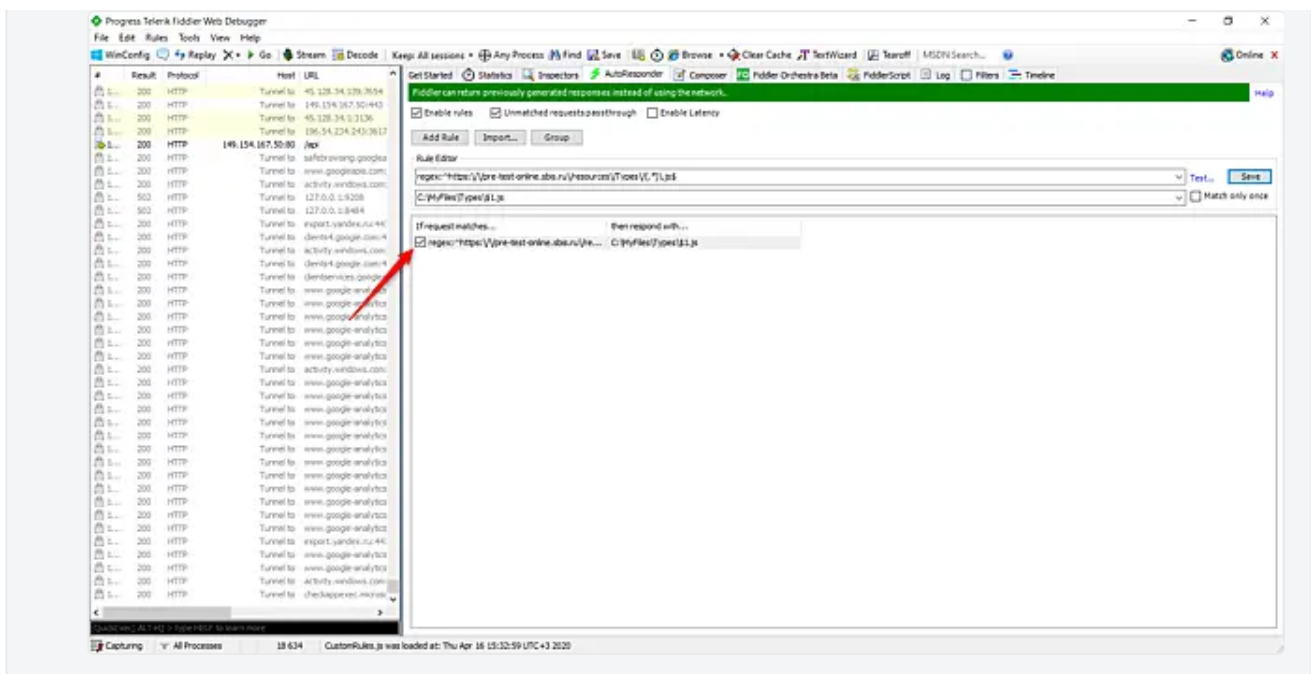
5. Если нужно заменить все файлы модуля, то правило необходимо изменить:

```
If request matches == regex:^https://test-
(online|cdn).sbis.ru/static/resources/Types/(.*)\\.\\w*(?:.*)?$
then respond with == C:\\MyFiles\\Types$2.$3
```

6. Чтобы проверить подпадает ли конкретный url под регулярное выражение, нажмите кнопку "Test".

7. После редактирования правила нажмите кнопку "Save".

 Для отмены правила достаточно снять чекбокс, расположенный напротив правила.



[Подробнее о решении проблем при настройке Fiddler.](#)

Для подмены настроек рэгех выполните следующие шаги:

1. Настройте сборку в вашем проекте через wasaby-cli.
2. Запустите сборку проекта.
3. Перейдите в папку, в которую был собран проект. По умолчанию проект собирается в папку application. Для изменения каталога сборки создайте секцию wasaby-cli в файле package.json.
4. Найдите в каталоге сборки файл [loadPageConfig.ts](#), который содержит метод loadPageData. Этот метод вызывается при SPA-переходе и обращается в back за получением рэгех-данных по regId страницы.
5. Замените data на новый объект рэгех.

```
const loadPageData = (
  pageId: string,
  Router: IRouter,
  changePageIdParams: object = {}
): Promise<ILoadSabyPageConfigResult> => {
  const configSource = new SbisService({
    endpoint: 'SiteNavigation',
  });
  return configSource
    .call('getPage', {
      Url: pageId || null,
    })
    .then((result) => {
      const data = result.getRow()?.getRowData() || ({} as ILoadSabyPageConfigResult);

      if (data.contentConfig?.changePageIdHandler) {
        return loadAsync<ChangePageIdHandler>(data.contentConfig.changePageIdHandler)
          .then((changePageIdHandler: ChangePageIdHandler) => {
            const changePageIdHandlerResult = changePageIdHandler(pageId, Router);
```

6. Замените файл loadPageConfig.ts с помощью Fiddler или Overrides.
7. Распакуйте модуль Page через WasabyDevTools и замените файл loadPageConfig.ts. Шаг 7 выполняется в случае быстрой проверки настройки конкретного поля из рэгех. В Chrome DevTools на вкладке **Source** найдите файл loadPageConfig.ts и метод loadPageData. Замените data при отладке файла, используя точки останова на методе SiteNavigation.getPage.